

2026-05-23 - Threads and Synchronization

Goal

Learn how to create and manage multiple threads in Zephyr and synchronize them using semaphores, mutexes, and message queues.

What you will learn

- How to define and start threads using `K_THREAD_DEFINE`
- How to use `k_sem`, `k_mutex`, and `k_msgq` for synchronization
- Thread priorities and cooperative vs preemptive scheduling
- How to pass data between threads safely

Overview

Zephyr is a preemptive RTOS. Each thread has a priority and a stack. Threads run concurrently and share resources, so synchronization primitives are needed to avoid race conditions.

Key concepts

Defining a thread:

```
K_THREAD_DEFINE(my_thread, 1024, my_thread_fn, NULL, NULL, NULL, 5, 0, 0);
```

Arguments: name, stack size, entry function, arg1, arg2, arg3, priority, options, delay.

Semaphore — signal between threads:

```
K_SEM_DEFINE(my_sem, 0, 1);  
k_sem_give(&my_sem);    // signal  
k_sem_take(&my_sem, K_FOREVER); // wait
```

Mutex — protect shared data:

```
K_MUTEX_DEFINE(my_mutex);  
k_mutex_lock(&my_mutex, K_FOREVER);  
// critical section  
k_mutex_unlock(&my_mutex);
```

Message queue — pass data between threads:

```
K_MSGQ_DEFINE(my_queue, sizeof(int), 10, 4);  
k_msgq_put(&my_queue, &value, K_NO_WAIT);  
k_msgq_get(&my_queue, &value, K_FOREVER);
```

Thread priorities

- Lower number = higher priority
- Negative priorities are cooperative (must yield explicitly)
- Priorities 0–14 are preemptive by default

Why this matters

Almost every real embedded application uses multiple threads: one for sensors, one for communication, one for UI. Understanding Zephyr threading is essential before building any non-trivial application.

Practice tasks

1. Create two threads at different priorities and observe which runs first.
2. Use a semaphore to have thread A trigger thread B.
3. Pass an integer counter from a producer thread to a consumer thread via `k_msgq`.

Example folder

See `src/2026-05-23_Threads_and_Synchronization` for the day's code example.

Next topic

Tomorrow we explore GPIO configuration and interrupt-driven input handling.